MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

Tools: Function Calling & MCP

How agents act on the world: declaring tools, executing calls, and standardising integration with the Model Context Protocol.

Unit 03 · Session 03 · Week 3

Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **Where we are** opening the ACT step of the agent loop
2. **Function calling mechanics** schemas, the request-execute-append cycle, tool choice, parallelism
3. **Failure handling** errors as observations, validation, retries, timeouts
4. **Designing tool interfaces** naming, descriptions, granularity - the agent-computer interface
5. **Model Context Protocol** the N x M problem, host/client/server, primitives, transports
6. **Security & ecosystem** the execution boundary; where MCP stands in 2026

Where we are: zooming into ACT

Session 02 gave us the loop - today we open its action step

Last week — the agent loop: reason, act, observe; state = the growing message list

Today — what 'act' actually means - how a tool call works, wire-level

Mechanics first — schema declaration, the request-execute-append cycle, failure handling

Then design — tool interfaces the model can actually use reliably

Then standardisation — the Model Context Protocol and its 2026 ecosystem

Lab 03 — one calculator wired twice: raw calls over ollama.chat, then a minimal MCP-style server

WHY TOOLS MATTER

Any agent that searches, fetches pages or writes files owns three integrations it must declare, execute and secure. Tools are where the agent meets the world.

1

Function calling mechanics

How a model 'calls' a function without ever calling anything

What a tool call really is

The model only ever emits text - structured text

Declaration — tool = name, description, JSON-Schema parameters; the minimal loop sends them with each request

Emission — the model outputs a structured call: tool name plus JSON arguments

Execution — a runtime parses, validates and runs the function - locally or remotely

Learned behaviour — models are post-trained to emit calls (Schick et al., 2023; OpenAI, 2023)

Strict mode — constrained decoding guarantees schema-valid JSON - not sensible or authorised arguments

KEY POINT

'Function calling' is a misnomer: the model writes a request to call a function. The runtime executes and holds tool authority; model text can still mislead or disclose.

Declaring a tool: the JSON Schema

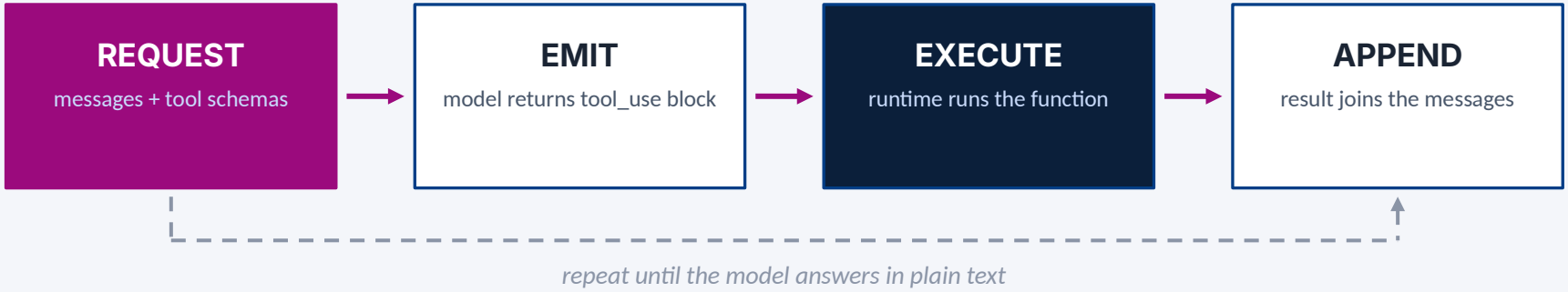
Name, description, parameters - Anthropic API style; OpenAI differs in field names and nesting (parameters vs input_schema)

calculator_tool.py

```
calculator_tool = {
    "name": "calculator",
    "description": (
        "Evaluate a basic arithmetic expression and "
        "return the numeric result. Use this for "
        "arithmetic instead of estimating."
    ),
    "input_schema": {
        "type": "object",
        "properties": {
            "expression": {
                "type": "string",
                "description": "e.g. '(17.4 - 3) * 2'"}},
        "required": ["expression"],
    },
}
```

The request-execute-append cycle

One HTTP round trip per reasoning step - the LLM stays stateless



Stop signal — stop_reason 'tool_use' vs 'end_turn' tells the runtime which branch to take

Pairing — each result carries the call's id - the model matches them by id

Parallel calls — one turn may hold several tool_use blocks; run them concurrently only if independent

Tool-choice modes: who decides?

Whether a tool gets used is a runtime parameter, set per request - not a model property

Mode	Model behaviour	Typical use
auto	decides freely: call a tool or answer	general agent loops
required / any	must call some tool, picks which	force a call; grounding still needs checking
specific tool	must call the named tool	structured extraction, routers
none	tools visible, plain text only	planning or summarisation steps

Orthogonal: parallel tool use - one turn may emit several calls; execute them (concurrently only if independent, with compatible side effects), then return all results before the next model call.

Error handling: failure as observation

A robust loop treats tool errors as data, not exceptions

Errors are data — return the failure as a tool result (is_error): a filtered message, not a raw stack trace

Validate first — schema validity, semantic validity and authorization are separate checks before executing

Retry budgets — cap attempts per tool and per run; backoff for rate limits; separate transport from domain errors

Idempotency — retry only where semantics permit; a write with unknown outcome needs an idempotency key or state check

Timeouts — a hung tool hangs the whole loop - every call needs a deadline

Escalation — after N failures, surface to the user instead of looping forever

DESIGN RULE

A good error message is actionable for the model: say what was wrong and what a valid call looks like. 'Error 500' teaches it nothing.

2

Designing good tool interfaces

The schema is a prompt: tool design is agent UX

The tool schema is a prompt

In the minimal loop the definitions ride along in every request - the model reads every word

Names — verb_noun and unambiguous: search_web, not handle_query

Descriptions — say what it does, when to use it - and when not to

Parameters — describe each field, give an example value, constrain with enums

Required fields — choose by domain need; implement defaults in runtime code - schema 'default' fills nothing

Output shape — return concise, structured results - the model pays tokens to read them

Evaluate — test tools with the model in the loop, not just unit tests (Anthropic, 2025)

ACI

Think 'agent-computer interface': tool design is interface design where the user is a language model under token pressure.

The same search tool, designed twice

Every weakness on the left becomes a wrong or wasted call

Weak interface

- name: doSearch3
- description: 'performs search'
- params q, t, n - undocumented
- returns raw HTML, 40,000 tokens
- on failure: stack trace, or silence

Strong interface

- name: search_web
- 'Search the web; use for recent facts'
- query described; max_results: 1-10
- returns title, URL, snippet per hit
- on failure: 'rate limited, retry in 20 s'

Granularity and ergonomics

Lessons from Anthropic's 'Writing effective tools for agents' (2025)

Granularity

Prefer a few task-shaped tools over mirroring a full API; consolidate multi-step chores into one call (schedule_meeting, not six endpoint wrappers).

Response design

Return what the next decision needs: ids, names, status. Offer a concise format; truncate huge payloads and hand back a pointer instead.

Namespacing

Prefix related tools (github_create_issue, slack_post_message) so dozens of tools from different sources stay distinguishable.

Token budget

In the minimal loop every schema is re-sent per request; fifty verbose tools cost thousands of tokens before any work. Larger systems load or search tools on demand.

3

From bespoke integrations to MCP

A protocol for the N x M problem

Why a protocol at all?

Without a standard: N x M bespoke integrations.

Every agent product re-implements the same GitHub, Slack and database connectors, once per model stack. A shared protocol turns N x M into N + M: each host speaks it once, each system exposes it once. (Anthropic, 2024)

MCP architecture: host, client, server

Introduced by Anthropic in November 2024; JSON-RPC underneath



Host — owns the model loop and the conversation; merges tools from many servers

Client — 1:1 with one server; since 2026-07-28 no protocol-level session - every request is self-describing

Server — any process, any language - wraps one capability behind the protocol

MCP primitives: who controls what

The control model is the point - not just a list of features

Tools

Model-controlled: functions the LLM decides to call - search, compute, write. Exactly the function-calling primitive, now standardised.

Resources

Application-controlled: readable data (files, tables, schemas) addressed by URI. The host decides what to attach as context; no side effects.

Prompts

User-controlled: parameterised templates the user invokes explicitly - slash commands in an IDE are the canonical example.

Client primitives

Since 2026-07-28 a server cannot call the client: it returns input_required (multi round-trip request) and the client retries with answers. Sampling and roots are deprecated.

JSON-RPC, transports, discovery

Spec revisions 2024-11-05 · 2025-03-26 · 2025-06-18 · 2025-11-25 · 2026-07-28 (stateless rewrite)

Messages — JSON-RPC 2.0 requests, results, notifications - stateless since 2026-07-28: version and capabilities ride in _meta

stdio transport — local subprocess over stdin/stdout; access = OS permissions plus credentials you hand it, no OAuth flow

Streamable HTTP — remote: one endpoint, Mcp-* headers, no session id; auth optional, OAuth 2.1 for protected servers

Lifecycle — no handshake; servers implement server/discover, clients may call it first; legacy initialize only if the server supports old versions

Discovery — tools/list returns schemas at runtime (with cache hints); tools/call invokes; subscriptions/listen notifies

WHY IT MATTERS

Discovery goes beyond function calling: a host can connect to a server it has never seen and learn its tools at runtime.

A calculator tool as an MCP server

Python SDK 2.x, MCPServer (formerly FastMCP) - schema generated from the signature; complete, runnable file

calc_server.py

```python
from mcp.server import MCPServer

mcp = MCPServer("calc")

@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two integers and return the sum."""
    return a + b

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Auto-schema — type hints + docstring become the JSON Schema

Why int — a typed contract; expression evaluators need a bounded, safe parser

stdio — the host launches it as a subprocess; pin the SDK version

Lab 03 — builds the same mechanism in plain Python, no SDK

One calculator, two wirings (Lab 03)

Local Ollama model (qwen2.5:7b), no API keys, offline corpus; gap-fill notebook, report tasks R1-R4

Parts B-E: raw function calling

- declare: name + description + JSON Schema
- request-execute-append over ollama.chat(tools=...)
- dispatch table: tool name to Python function
- validate args; failures return as observations
- search, read, compute over an offline corpus

Part F: MCP-style server + host

- plain Python, JSON-RPC dicts, no SDK
- schema derived from signature + docstring
- host discovers tools via tools/list
- add a tool (R3): host code stays unchanged
- R4: when is the protocol layer worth it?

4

Security & ecosystem

Who executes, who is exposed - and where MCP stands in 2026

The security boundary: who actually acts

Draw the trust line through the runtime, not the model

The model proposes — it can only ever emit structured text - it executes nothing

The runtime disposes — it enforces tool authority: credentials, validation, allowlists, confirmation gates

Still model-side — plain text can disclose data or mislead users; model errors raise the failure rate

Injection path — tool results are untrusted input - a fetched page can steer the next call

MCP twist — local servers execute on the host; remote servers add a separate service and trust boundary

Least privilege — narrow tokens, read-only modes, sandboxed execution (Sessions 10, 13)

THREE QUESTIONS

How likely is an error (model)? What is it allowed to do (runtime authority)? What could it damage (tool impact)? Sessions 10 and 13 keep the same split.

MCP in 2026: an honest status

A real standard - and a real attack surface

Adopted

- OpenAI, Google DeepMind, Microsoft (2025)
- native in major IDEs and desktop hosts
- official registry, thousands of servers
- Linux Foundation (Agentic AI Foundation) since Dec 2025
- de-facto standard for agent-tool integration

Open problems

- server quality varies wildly, little vetting
- tool poisoning, rug pulls (Invariant Labs, 2025)
- remote auth still rough in practice
- many servers = context bloat, tool confusion
- spec churn: five revisions, core rewritten July 2026

TAKEAWAYS

What you should remember

- **Mechanics —** the model emits a structured call; the runtime executes and appends the result
- **Control —** tool-choice modes and parallel calls shape the loop's behaviour
- **Errors —** failures go back as observations - write messages a model can act on
- **Interface design —** schemas are prompts; naming, descriptions and granularity drive reliability
- **MCP —** host-client-server protocol that turns N x M integrations into N + M
- **Security —** the runtime enforces tool authority; separate error probability, authority and impact

NEXT SESSION Prompt engineering for agents - system prompts, few-shot, chain-of-thought.

Reading & references

- Schick et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.
- Patil et al. (2023). Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.
- OpenAI (2023). Function calling and other API updates. openai.com, June 13, 2023.
  The announcement that made tool use an API primitive.
- Anthropic (2024). Introducing the Model Context Protocol. anthropic.com, November 2024.
- Model Context Protocol Specification, revision 2026-07-28. modelcontextprotocol.io.
  Read: Architecture, Tools, Transports, and the changelog (stateless core, multi round-trip requests).
- Anthropic Engineering (2025). Writing effective tools for agents.
  Source for the interface-design guidance in section 2.
- Invariant Labs (2025). MCP Security Notification: Tool Poisoning Attacks.
  Background for the supply-chain discussion.